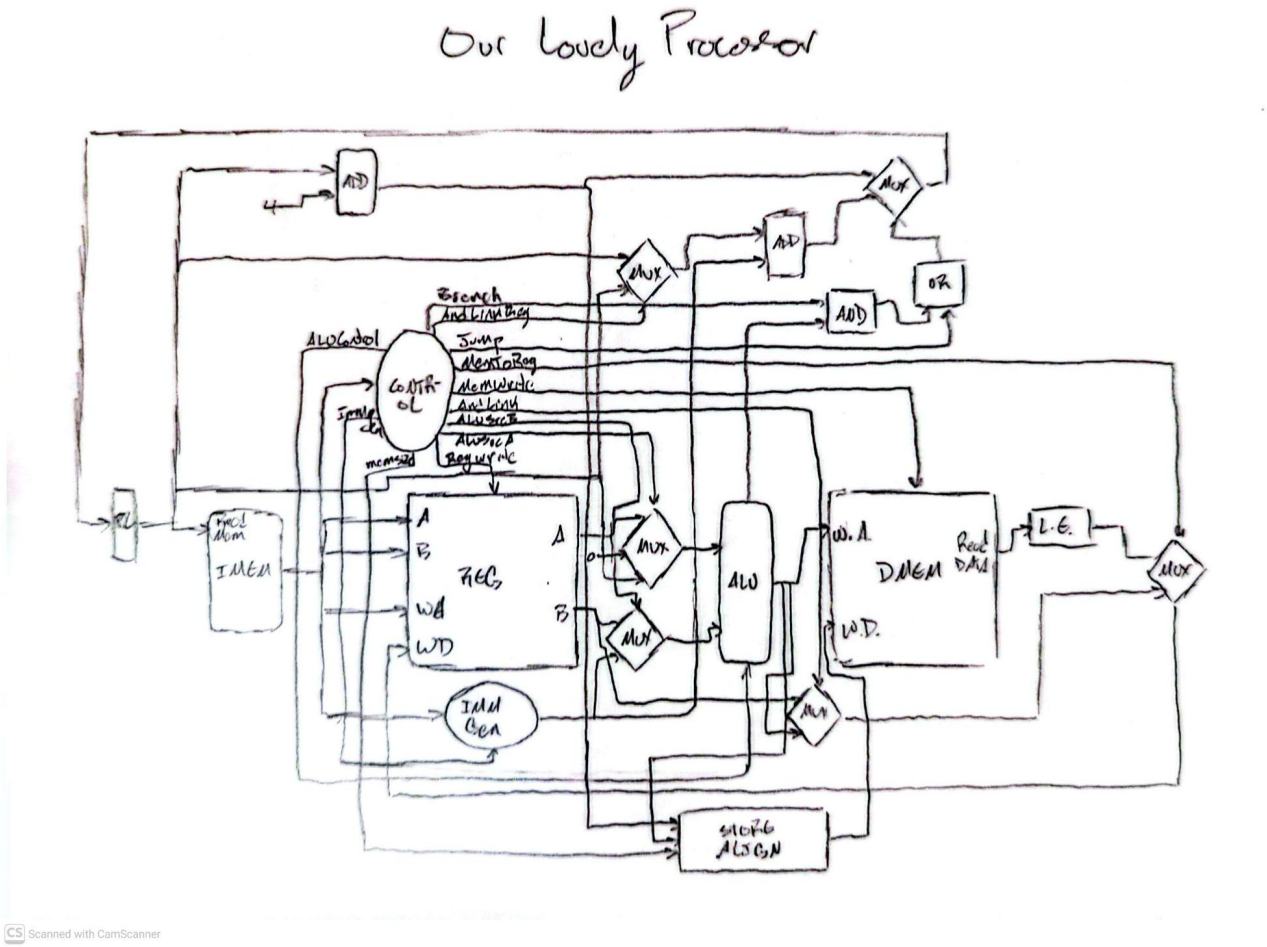
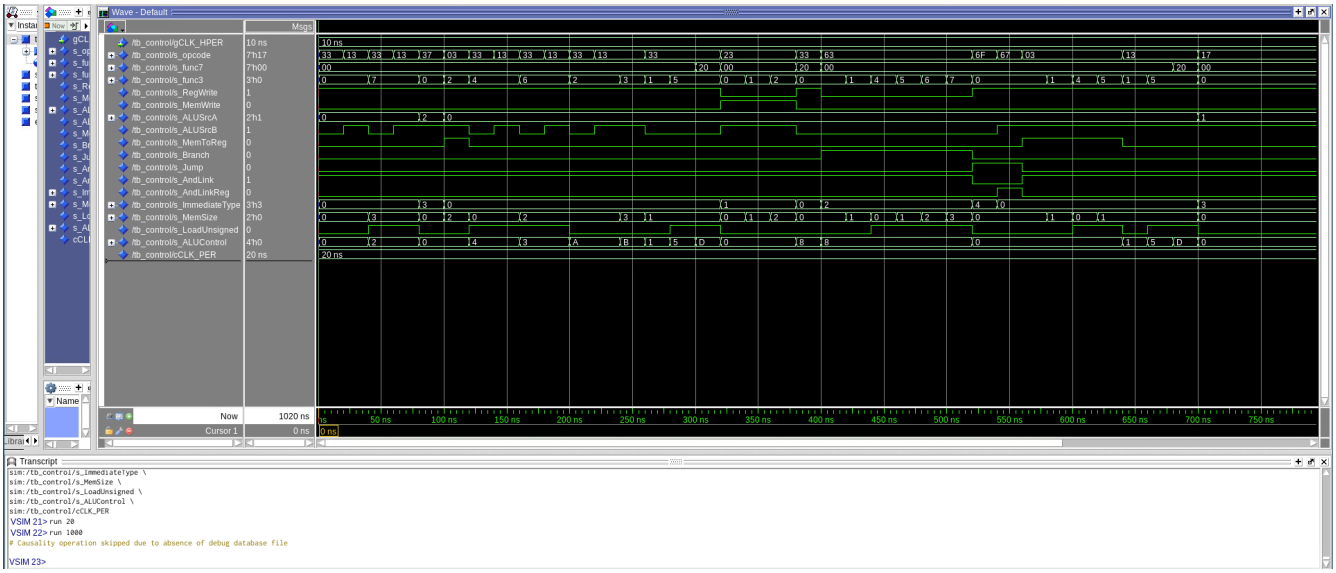
CPRE 3810 Final Project Part 1
Troy Tomson and Isaiah Steele

2.d:



3.1.a: *Complete and included with submission.*

3.1.b:



Here is an image of us running our control testbench. We have asserts set up for each instruction with the values currently in the csv like so:

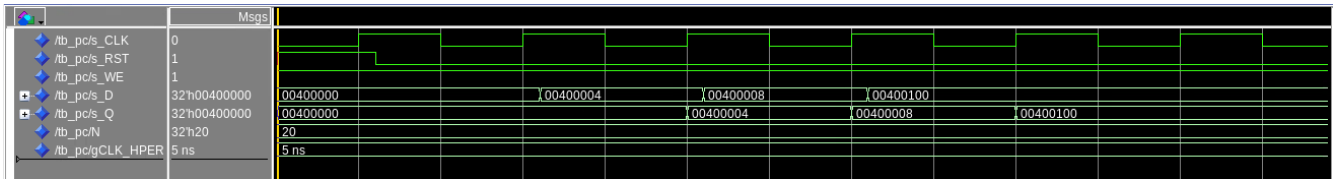
```
95      -- 1 add -- r type
96      s_opcode <= "0110011"; s_func3 <= "000"; s_func7 <= "0000000";
97      wait for cCLK_PER;
98      -- expected: regwrite=1 memwrite=0 alusrca=00 alusrcb=0 memtoreg=0 branch=0 jump=0 andlink=1 andlinkreg=0 immtype=000 alucontrol=0000
99      assert (s_RegWrite = '1' and s_MemWrite = '0' and s_ALUSrcA = "00" and s_ALUSrcB = '0' and s_MemToReg = '0' and s_Branch = '0' and s_Jump = '0' and
100       report "problem on add" severity error;
```

Using this strategy, if there is a discrepancy between expected and real values, we will get an alert in the transcript box at the bottom of the screenshot. We do not get any messages in the transcript (see waveform screenshot), and so there are no current discrepancies.

3.2.a:
The PC should (in the typical case) be incremented by 4 for each command. Additionally, in the case of a j, jal, or jalr register it should set the PC to the value of the immediate. For branch instructions, the branch condition must resolve true to update the address to the address of the function it must branch to.

3.2.b:
See digital diagram for more. Fetch logic is heavily integrated into the design of the processor. All instructions start with the PC. 4 is added in the default case (proceed to the next instruction). As the current PC is passed into the instruction memory. This path is selected with the exceptions of a jump/jal/jalr command, or a branch command if the branch condition is met. In that case, the value from the immediate generation is added to the current PC value, and that result is put back into the PC.

3.2.c:



This is the waveform of the testbench file for the pc. This tests, in order, that: the pc resets, the pc loads an initial address, the pc increments (twice), and that the pc loaded a branch loads a branch/jump target. Those tests are written clearly here for reference with the waveform:

```
52          assert s_Q = x"00400000"
53              report "FAIL: PC reset value is incorrect"
54              severity error;
55
56          s_RST <= '0';
57
58          -- Load the initial PC
59          s_D <= x"00400000";
60          s_WE <= '1';
61          wait until rising_edge(s_CLK);
62          wait for 1 ns;
63
64          assert s_Q = x"00400000"
65              report "FAIL: PC did not load initial address"
66              severity error;
67
68          -- PC + 4
69          s_D <= x"00400004";
70          wait until rising_edge(s_CLK);
71          wait for 1 ns;
72
73          assert s_Q = x"00400004"
74              report "FAIL: PC did not increment to 0x00400004"
75              severity error;
```

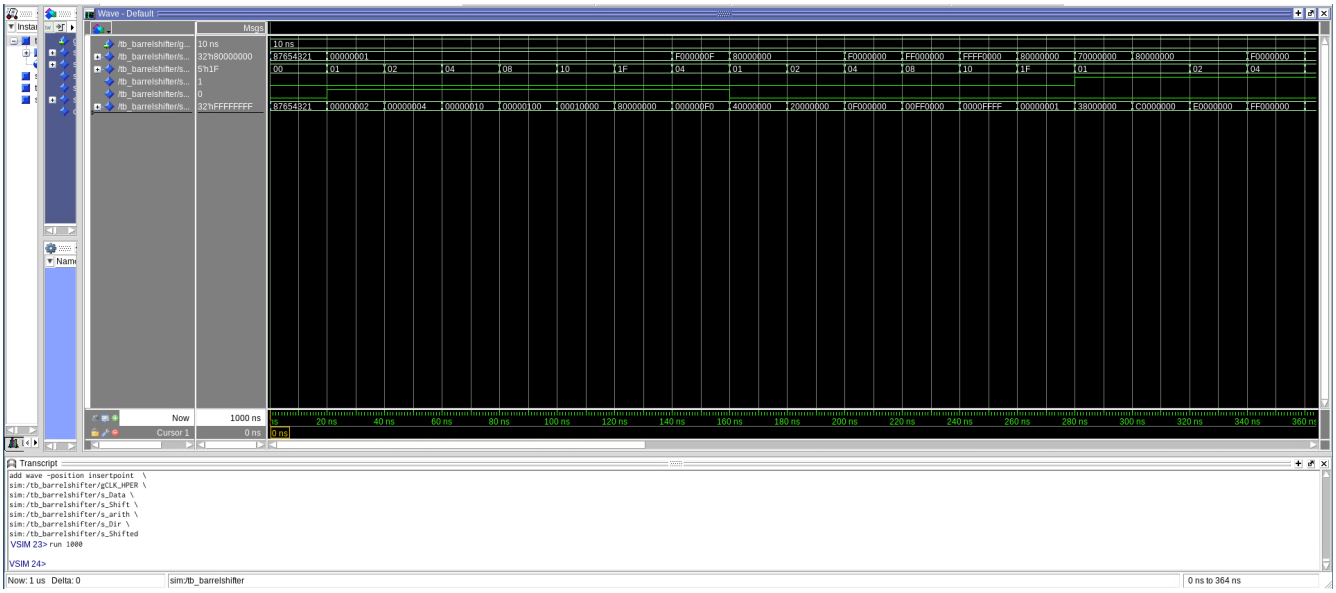
```
77          -- PC + 4 again
78          s_D <= x"00400008";
79          wait until rising_edge(s_CLK);
80          wait for 1 ns;
81
82          assert s_Q = x"00400008"
83              report "FAIL: PC did not increment to 0x00400008"
84              severity error;
85
86          -- Branch/Jump
87          s_D <= x"00400100";
88          wait until rising_edge(s_CLK);
89          wait for 1 ns;
90
91          assert s_Q = x"00400100"
92              report "FAIL: PC did not load branch/jump target"
93              severity error;
```

3.3.1.a:
sra is sign extended while srl always pads the rest of the word with 0's. Sra is intended for use with signed integers, while srl is for unsigned integers. Because sla would just pad the left bits with 0's, it's effectively the same as a sll.

3.3.1.b:
The shifter works by taking in a 5 bit input and shifting the register based on the input. First by 16, then 8, 4, 2, and 1 bits which allow for 31 bits of movement. There's also a left/right and arithmetic selector which just fills the remaining bits at the end of the shifting with 1's if the value being shifted is negative.

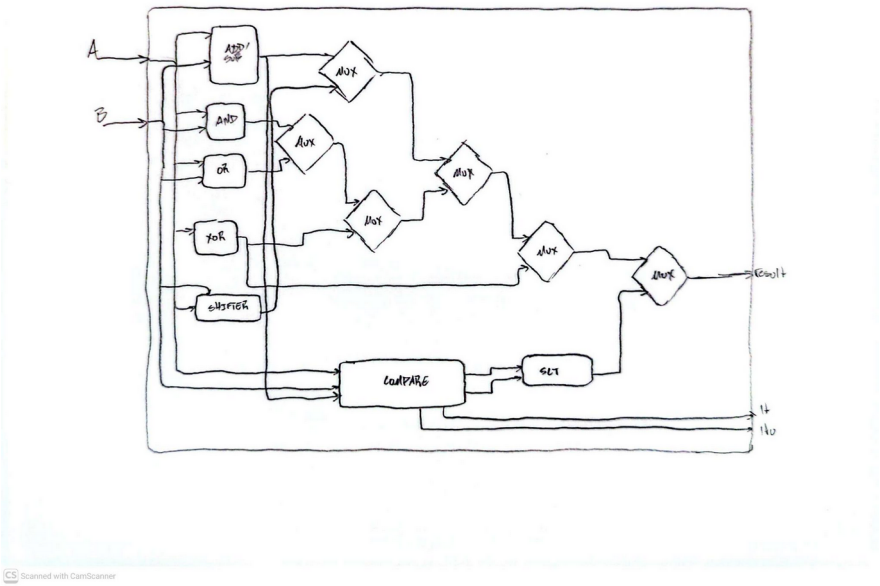
3.3.1.c:
You can simply check for a shift right bit and move in the opposite direction if it's enabled. This was done by using an if statement in my code.

3.3.1.d:



This is the waveform of our testbench that tests the barrel shifter. Because we use assert statements in the vhdl, and we get no errors in our transcript box, we know that our tests have passed. These tests cover shifting amounts, and logical/arithmetic shifts. The barrel shifter doesn't need to know if an immediate is being used, so the shifter only needs to show correct shifts of up to 32 in distance, and do sll, srl, and sra to cover all shifts.

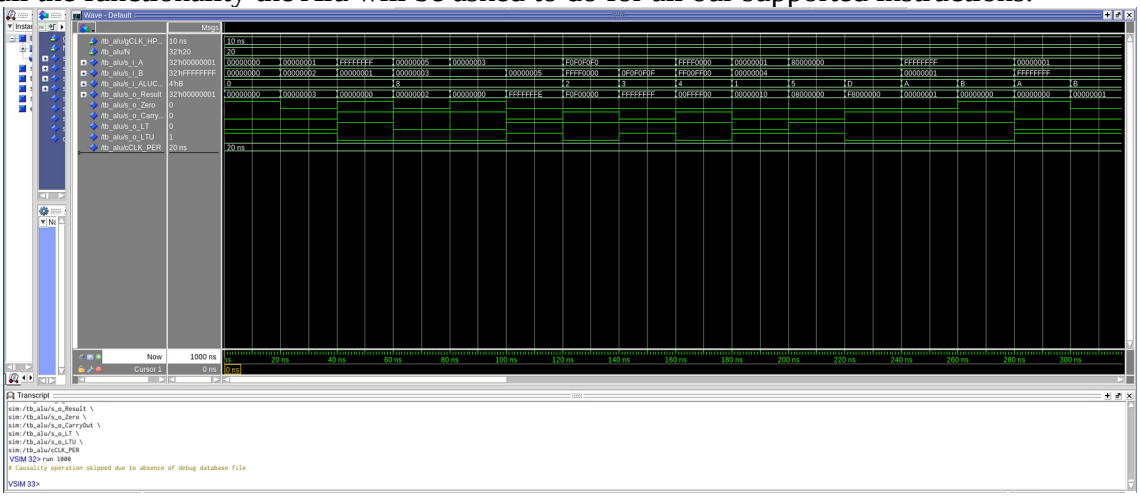
3.3.3:



The ALU will produce zero (forgot to include the zero output here) by checking to see if the result value is all zeros. If so, the zero output will be set. SLT is computed by using the add/sub output to decide if a is less than b (accounting for operand signs, so A and B are going into the compare part too) and that output is fed into slt which turns the 1-bit output into a 32-bit output (. . . . 001 or . . . . . 000).

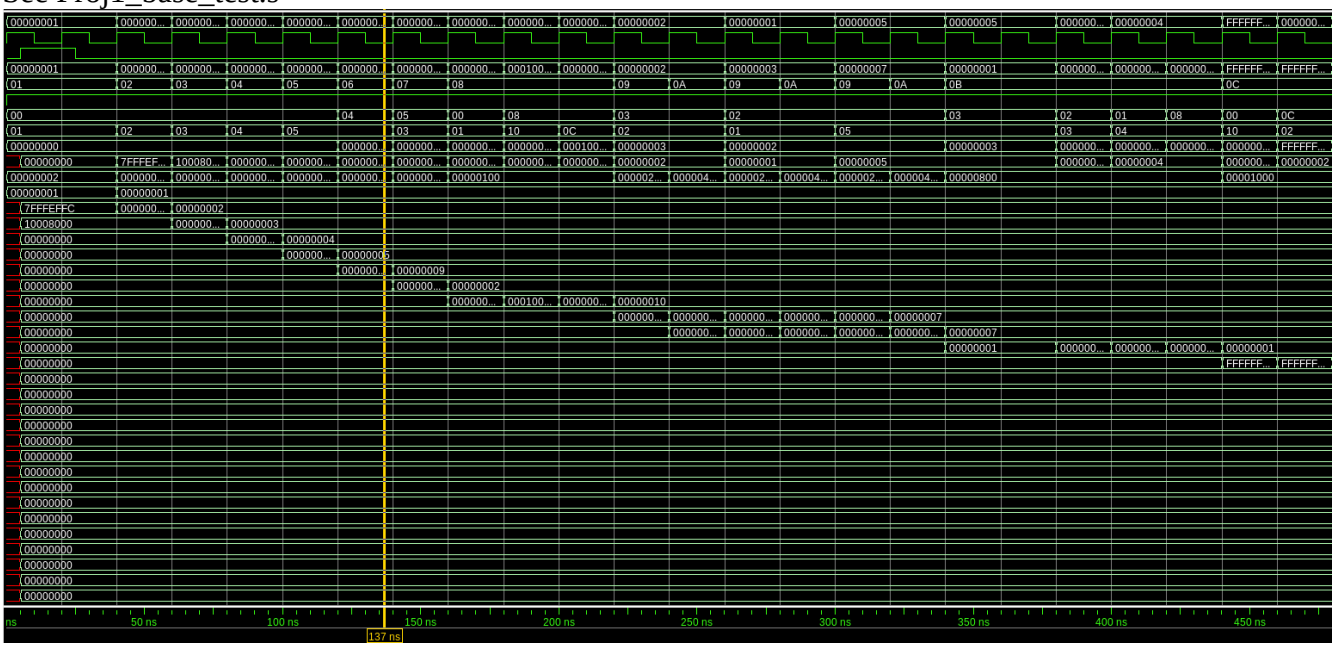
3.3.5: / 3.3.8:

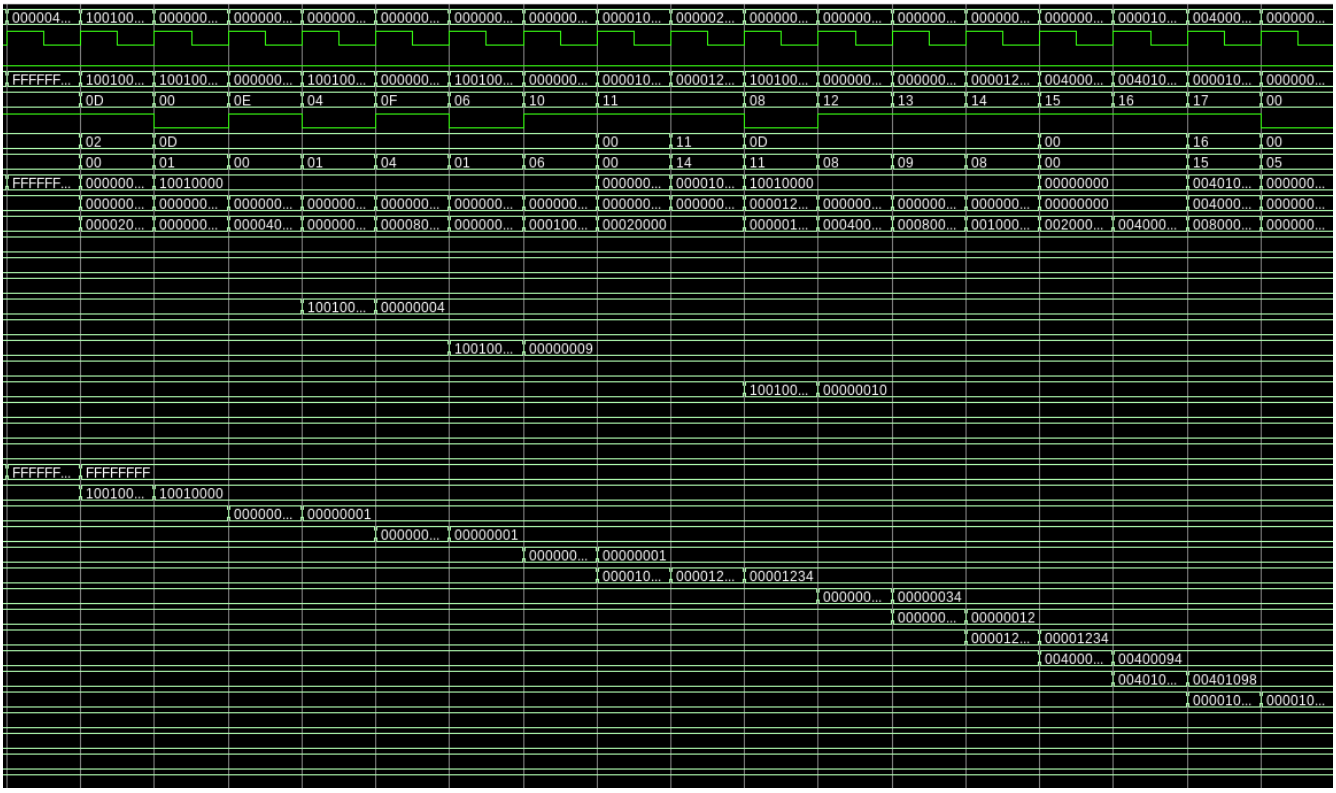
Work for 3.3.8 was completed before the 3.3.5 portion of the report was written, so they will be combined. This waveform tests the use of add, sub, and, or, xor, sll, srl, sra, slt, and sltu. The inpts for A and B are provided along with the ALU_Control line, and the ALU executes each instruction. This test covers all the functionality the Alu will be asked to do for all our supported instructions.



4.a:

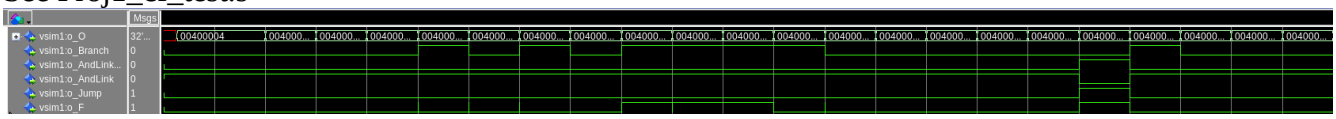
See Proj1_base_test.s





The first 5 instructions just add vals 1-5 to reges 1-5.
Next is an add, sub, another addi, a right shift, and a left shift. All match up correctly with the expected values.
Comparisons with and/or/xor and their immediate versions are also correctly matching their expected values.
The set comparisons are also working as intended, along with loads and stores. Essentially, all of the values in the registers are the same as the expected ones when walking through/making the test program manually.

4.b:
See Proj1_cf_test.s



It branches when branch is true AND the beq resolves to true. The last 5 instructions repeat for 5 layers. Everything matches up with the comments I put into the testbench when developing it.

4.c:
see Proj1_mergesort.s
The contents of the memory produce a sorted list (in order of ascending value).

vsim2:clk	0						
vsim2:addr	10'h000	027	3D7	3DE	3DD	3DC	3DB	3DA
vsim2:data	32'h01010101	004000...	00000000	0040009C	10010004	00000000	00000001	00000002
vsim2:be	4'h2	F	1	F				
vsim2:we	0							
vsim2:q	32'h0000000C	00000000		00000000	00000000	00000000	00000000	00000000
vsim2:q_local	32'h0000000C	00000000		00000000	00000000	00000000	00000000	00000000
vsim2:i_A	32'h00000000	000000...	7FFFEF7C	7FFFEF5C				
vsim2:i_B	32'h00000001	004000...	FFFFFFE0	0000001C	00000018	00000014	00000010	0000000C

Here is a small snippet. The testbench successfully accesses, retrieves, (and can store, as the data was originally stored in the memory by my program). The actual program is quite complex, but all other operations (such as comparison, arithmetic operations, pc operations, etc) are all functional.

5: The maximum frequency of our processor is 34.25 megahertz. In order to speed up the processor, our time would be best used optimizing the memory if possible, looking at the ALU (specifically shifting), or looking at the register file. Those are our worst slowdowns, and they're some of the most-used (or always used) portions of the processor. A speedup there would help the speed of the entire system.

Below is the labeled image of the critical path of our processor: PC – Imem – Reg File – ALU – Dmem – Load Extender – Mux – Reg File

